Embedded systems Architecture (BACSE324)

Lab Sl. No.	**8**	
Lab Slot	**L45+ L46**	
Registration No.	**25BCE2634**	
Name:	**KETAN MULANI**	
Mobile no.:	**+91 9929129845**	
Title::		
TRA Name:		**Assigned date of Task :**
TRA Mobile number:		**Date of Completion:**
Faculty Name:	**DEBASHISH DASH**	**Assessment no: 8**
Email ID of student : ketan.mulani2025@vitstudent.ac.in		
VTOP uploading Date and time:		

Contents

Component	Page No.	Remarks
Question	**3,8,13,20**	
AIM of Experiment	**3,8,13,20**	
Tools Required	**3,8,13,20**	
Basic Brief Theory	**3,8,13,20**	
Circuit Connections	**4,9,14,21**	
Arduino Code with Proper Comments	**4,9,10,14,15,22**	
Procedure to perform the AIM	**5,10,16,23**	
Output Snips with proper path	**6,11,17,18,24**	
Results	**7,12,19,25**	
Inference	**7,12,19,25**	

Signature of TRA

Program 1 :

Design a hardware circuit using physical Arduino board for blinking a LED.

Aim of the Experiment :

To design and build a hardware circuit using a physical Arduino Uno board to blink an LED connected to digital pin 13, with the LED staying ON for 1 second and OFF for 1 second repeatedly.

Tools Required:

1. Arduino Uno board (physical)
2. USB cable (Type A to Type B)
3. 5 mm LED
4. 220 Ω resistor
5. Breadboard
6. Jumper wires
7. Computer with Arduino IDE

Brief Theory:

A Light Emitting Diode (LED) is a PN-junction diode that emits light when it is forward biased. It has two terminals: the longer leg is the anode (+) and the shorter leg is the cathode (−). An LED must always be connected in series with a current-limiting resistor, otherwise it can draw excessive current and get damaged.

The digital pins of the Arduino Uno give an output of 5 V when set HIGH and 0 V when set LOW. The recommended current from a pin is 20 mA (absolute maximum 40 mA). In this experiment, pin 13 is configured as an OUTPUT using pinMode(). The function digitalWrite() switches the LED ON (HIGH) or OFF (LOW), and delay(1000) holds each state for 1000 ms. Repeating this sequence inside loop() makes the LED blink continuously.

Circuit Connections:

Component Pin	Arduino Uno Pin
LED Anode (+) via 220 Ω resistor	Digital Pin 13
LED Cathode (−)	GND

Arduino Code:

```
// Define the Arduino pin connected to the LED
const int ledPin = 13;

void setup() {
  // Set the LED pin as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Turn the LED ON
  digitalWrite(ledPin, HIGH);

  // Wait for 1 second
  delay(1000);

  // Turn the LED OFF
  digitalWrite(ledPin, LOW);

  // Wait for 1 second
  delay(1000);
}
```

Manual calculation:

None

Procedure:

1. Connect the Arduino Uno board to the computer using a USB cable.
2. Place the LED and the 220 Ω resistor on the breadboard.
3. Connect the anode (longer leg) of the LED to digital pin 13 of the Arduino through the 220 Ω resistor.
4. Connect the cathode (shorter leg) of the LED to the GND pin of the Arduino.
5. Open the Arduino IDE and enter the Arduino code.
6. Define the LED pin and set it as an output using pinMode().
7. Turn the LED ON and OFF using digitalWrite() with a delay of 1000 ms.
8. Select the board as Arduino Uno and the correct COM port, then upload the code.
9. Observe that the LED blinks with 1 second ON and 1 second OFF.
10. Take a colour photograph of the hardware setup with the GPS location marked.

Result:

The LED blinking circuit was successfully designed and built using a physical Arduino Uno board. The LED connected to digital pin 13 blinked with an ON time of 1 second and an OFF time of 1 second.

Inference:

The experiment demonstrates that a digital output pin of the Arduino Uno can be used to control an LED. The ON/OFF timing of the LED is controlled by the delay() function, and a series resistor is necessary to limit the LED current to a safe value.

Program 2 :

Design a basic hardware circuit for a servo motor which will rotate for 0 to 180 degrees.

Aim of the Experiment :

To design and build a basic hardware circuit to interface a servo motor with Arduino Uno and rotate the servo shaft from 0° to 180° and back to 0°.

Tools Required:

1. Arduino Uno board (physical)
2. USB cable (Type A to Type B)
3. Micro servo motor (SG90)
4. Breadboard
5. Jumper wires
6. Computer with Arduino IDE

Brief Theory:

A servo motor is a rotary actuator that contains a DC motor, a gear train, a position-feedback potentiometer and a control circuit. It allows the shaft to be moved to a required angle and held there. A standard hobby servo has three wires: red (VCC, 5 V), brown or black (GND) and orange or yellow (control signal).

The shaft position is set by a Pulse Width Modulation (PWM) signal repeated every 20 ms (50 Hz). A pulse width of about 1 ms moves the shaft to 0°, about 1.5 ms to 90° and about 2 ms to 180°. The Arduino Servo library generates these pulses automatically when the function write(angle) is called. In this experiment, the servo signal wire is connected to digital pin 9. The program increases the angle from 0° to 180° in steps of 1° with a 15 ms delay so that the shaft can reach each position, and then decreases the angle from 180° back to 0°.

Circuit Connections:

Servo Motor Wire	Arduino Uno Pin
VCC (Red)	5V
GND (Brown/Black)	GND
Signal (Orange/Yellow)	Digital Pin 9

Arduino Code:

```
#include <Servo.h>

// Create a Servo object to control the servo motor
Servo myServo;

// Define the Arduino pin connected to the servo signal wire
const int servoPin = 9;

// Variable to store the current angle of the servo
int angle = 0;

void setup() {
  // Attach the servo motor to digital pin 9
  myServo.attach(servoPin);
}

void loop() {
  // Rotate the servo from 0 to 180 degrees
  for (angle = 0; angle <= 180; angle++) {
    myServo.write(angle);
    // Wait 15 ms for the servo to reach the position
    delay(15);
  }

  // Rotate the servo back from 180 to 0 degrees
```

```
  for (angle = 180; angle >= 0; angle--) {
    myServo.write(angle);
    // Wait 15 ms for the servo to reach the position
    delay(15);
  }
}
```

Manual calculation:

None

Procedure:

1. Connect the Arduino Uno board to the computer using a USB cable.
2. Connect the VCC (red) wire of the servo motor to the 5V pin of the Arduino.
3. Connect the GND (brown/black) wire of the servo motor to the GND pin of the Arduino.
4. Connect the signal (orange/yellow) wire of the servo motor to digital pin 9 of the Arduino.
5. Open the Arduino IDE and include the Servo library.
6. Create a Servo object and attach it to pin 9 using attach().
7. Rotate the servo from 0° to 180° and back to 0° using write() inside for loops with a delay of 15 ms.
8. Select the board as Arduino Uno and the correct COM port, then upload the code.
9. Observe that the servo shaft rotates from 0° to 180° and returns to 0° continuously.
10. Take a colour photograph of the hardware setup with the GPS location marked.

Result:

The servo motor circuit was successfully designed and built using a physical Arduino Uno board. The servo shaft rotated smoothly from 0° to 180° and back to 0° continuously.

Inference:

The experiment demonstrates that the position of a servo motor can be controlled by the Arduino using PWM signals through the Servo library. The angle of the shaft depends on the width of the control pulse, and the delay in the program sets the speed of rotation.

Program 3 :

Design a basic hardware circuit for interfacing an ultrasonic sensor with Arduino uno.

Aim of the Experiment :

To design and build a basic hardware circuit to interface an HC-SR04 ultrasonic sensor with Arduino Uno, measure the distance of an object and display it on the Serial Monitor.

Tools Required:

1. Arduino Uno board (physical)
2. USB cable (Type A to Type B)
3. HC-SR04 ultrasonic distance sensor
4. Breadboard
5. Jumper wires
6. An object (box or book) to act as an obstacle
7. Computer with Arduino IDE and Serial Monitor

Brief Theory:

The HC-SR04 is an ultrasonic distance sensor that works at 5 V. It has a transmitter and a receiver for 40 kHz ultrasonic sound waves and four pins: VCC, TRIG, ECHO and GND. It can measure distances from about 2 cm to 400 cm.

To start a measurement, the Arduino sends a HIGH pulse of 10 µs to the TRIG pin. The sensor then transmits a burst of ultrasonic waves. When the waves are reflected by an object, the ECHO pin goes HIGH for a time equal to the round-trip travel time of the sound. This time is measured in the Arduino using the pulseIn() function. Distance = (Time × Speed of sound) / 2. The speed of sound is about 343 m/s, which is 0.0343 cm/µs. The result is divided by 2 because the sound travels to the object and back. The measured distance is displayed on the Serial Monitor at 9600 baud.

Circuit Connections:

HC-SR04 Pin	Arduino Uno Pin
VCC	5V
TRIG	Digital Pin 9
ECHO	Digital Pin 10
GND	GND

Arduino Code:

```
// Define the Arduino pins connected to the ultrasonic sensor
const int trigPin = 9;
const int echoPin = 10;

// Variables to store the echo time and the distance
unsigned long duration;
float distanceCm;

void setup() {
  // Start serial communication at 9600 baud
  Serial.begin(9600);

  // Set the trigger pin as an output
  pinMode(trigPin, OUTPUT);

  // Set the echo pin as an input
  pinMode(echoPin, INPUT);
}

void loop() {
  // Make sure the trigger pin is LOW
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
```

```
  // Send a HIGH pulse of 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Measure the time for which the echo pin stays HIGH
  duration = pulseIn(echoPin, HIGH, 30000UL);

  if (duration == 0) {
    // No echo received within the time limit
    Serial.println("No object detected / Out of range");
  } else {
    // Convert the echo time into distance in centimetres
    distanceCm = duration * 0.0343 / 2.0;

    // Display the distance on the Serial Monitor
    Serial.print("Distance: ");
    Serial.print(distanceCm, 1);
    Serial.println(" cm");
  }

  // Wait for 500 milliseconds before the next reading
  delay(500);
}
```

Manual calculation:

None

Procedure:

1. Connect the Arduino Uno board to the computer using a USB cable.
2. Place the HC-SR04 ultrasonic sensor on the breadboard.
3. Connect the VCC and GND pins of the sensor to the 5V and GND pins of the Arduino.
4. Connect the TRIG pin of the sensor to digital pin 9 and the ECHO pin to digital pin 10 of the Arduino.
5. Open the Arduino IDE and enter the Arduino code.
6. Initialize serial communication using Serial.begin(9600);.
7. Select the board as Arduino Uno and the correct COM port, then upload the code.
8. Open the Serial Monitor and set the baud rate to 9600.
9. Place an object in front of the sensor at different distances.
10. Observe the measured distance displayed on the Serial Monitor.
11. Take a colour photograph of the hardware setup with the GPS location marked.

Result:

The HC-SR04 ultrasonic sensor was successfully interfaced with the Arduino Uno. The distance of the object placed in front of the sensor was measured and displayed correctly on the Serial Monitor.

Inference:

The experiment demonstrates that an ultrasonic sensor can measure distance by calculating the time taken by sound waves to travel to an object and return. The Arduino generates the trigger pulse, measures the echo time and converts it into distance, which can be used in applications such as obstacle detection and parking systems.

Program 4 :

Design a hardware interfacing circuit for interfacing a normal 16*2 LCD as well as potentiometer and print your name and registration number.

Aim of the Experiment :

To design and build a hardware interfacing circuit to connect a 16 × 2 LCD (without I2C module) and a potentiometer with Arduino Uno, and to print the name and registration number on the LCD.

Tools Required:

1. Arduino Uno board (physical)
2. USB cable (Type A to Type B)
3. 16 × 2 LCD display (HD44780)
4. 10 kΩ potentiometer
5. 220 Ω resistor
6. Breadboard
7. Jumper wires
8. Computer with Arduino IDE

Brief Theory:

A 16 × 2 LCD can display 16 characters in each of its 2 rows (32 characters in total). It uses the HD44780 controller and has 16 pins: VSS, VDD, V0, RS, RW, E, D0 to D7, A (backlight +) and K (backlight −). The RS pin selects between command mode (LOW) and data mode (HIGH), the RW pin selects read or write (connected to GND for write only), and the E pin enables the data transfer.

The LCD can be operated in 4-bit mode, in which only the data pins D4 to D7 are used along with RS and E. This needs only 6 Arduino pins. The potentiometer is connected as a voltage divider and its middle pin (wiper) is connected to the V0 pin of the LCD to adjust the contrast of the display.

Circuit Connections:

LCD / Potentiometer Pin	Connection
VSS (Pin 1)	GND
VDD (Pin 2)	5V
V0 (Pin 3)	Potentiometer middle pin (wiper)
RS (Pin 4)	Digital Pin 12
RW (Pin 5)	GND
E (Pin 6)	Digital Pin 11
D0 – D3 (Pins 7 – 10)	Not connected
D4 (Pin 11)	Digital Pin 5
D5 (Pin 12)	Digital Pin 4
D6 (Pin 13)	Digital Pin 3
D7 (Pin 14)	Digital Pin 2
A (Pin 15)	5V through 220 Ω resistor
K (Pin 16)	GND
Potentiometer one end	5V
Potentiometer other end	GND

Arduino Code:

```
// Include the LiquidCrystal library for the 16x2 LCD
#include <LiquidCrystal.h>

// Initialize the LCD with the pins connected to RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  // Set up the LCD with 16 columns and 2 rows
  lcd.begin(16, 2);

  // Clear the display
  lcd.clear();

  // Print the name in the first row
  lcd.setCursor(0, 0);
  lcd.print("Ketan Mulani");

  // Print the registration number in the second row
  lcd.setCursor(0, 1);
  lcd.print("25BCE2634");
}

void loop() {
  // Nothing to repeat. The text stays displayed on the LCD.
  // Rotate the potentiometer to adjust the contrast.
}
```

Manual calculation:

None

Procedure:

1. Connect the Arduino Uno board to the computer using a USB cable.
2. Place the 16 × 2 LCD and the potentiometer on the breadboard.
3. Connect LCD pins VSS and RW to GND and pin VDD to 5V.
4. Connect the two outer pins of the potentiometer to 5V and GND and its middle pin to the V0 pin of the LCD.
5. Connect LCD pins RS and E to digital pins 12 and 11 of the Arduino respectively.
6. Connect LCD pins D4, D5, D6 and D7 to digital pins 5, 4, 3 and 2 of the Arduino respectively.
7. Connect the backlight pin A to 5V through the 220 Ω resistor and pin K to GND.
8. Open the Arduino IDE and include the LiquidCrystal library.
9. Initialize the LCD using lcd.begin(16, 2) and print the name and registration number using lcd.print().
10. Select the board as Arduino Uno and the correct COM port, then upload the code.
11. Rotate the potentiometer to adjust the contrast.
12. Observe the name and registration number displayed on the LCD.
13. Take a colour photograph of the hardware setup with the GPS location marked.

Result:

The 16 × 2 LCD and the potentiometer were successfully interfaced with the Arduino Uno. The name "Ketan Mulani" and the registration number "25BCE2634" were displayed on the LCD, and the contrast could be adjusted using the potentiometer.

Inference:

The experiment demonstrates that a 16 × 2 LCD can be interfaced with the Arduino Uno in 4-bit mode using only 6 digital pins and the LiquidCrystal library. The potentiometer controls the contrast voltage at the V0 pin, which is necessary to make the characters clearly visible.

ESA Lab Assessment report

Sl.no.	Components	Marks Obtained
1	**Pre Lab Work (Flow chart and Algorithm) documentation (2)**	
2	**Hardware or software Implementation (2)**	
3	**Manual calculation , Output data table / graph verification (2)**	
4	**Result with Inference (2)**	
5	**Post Lab Work (C Code with comments) documentation (2)**	
	Total (10)	

1	**Name: Ketan Mulani**	
2	**Reg. No.: 25BCE2634**	
3	**Lab slot: L 45+ L46**	**Lab. Slot Time: 3:50 PM – 5:30 PM**
4	**Assessment no.:8**	
5		
6	**Date of Assessment assigned**	
7	**Completion date**	
8	**VTOP Upload deadline**	
9	**Mobile number: +91 9929129845**	
10	**Email id: ketan.mulani2025@vitstudent.ac.in**	
11	**TRA Signature with Date**	
12	**Faculty Signature with Date**	